

RASTA-AI

Full Stack Engineering Document

How the AI-Based Smart Logistics and Accessibility Intelligence Platform for the North Eastern Region is built, and how a single trip flows through every layer of it — from the manager's plan to the driver's turn instruction and back.

PROBLEM STATEMENT	SIH26002 — AI-Based Smart Logistics and Accessibility Intelligence Platform for North Eastern Region (NER)
ORGANISATION	Ministry of Development of North Eastern Region (MDoNER)
THEME / CATEGORY	Smart Automation · Software
PRODUCT	RASTA-AI — Route Accessibility, Safety, Terrain & Analytics
RELEASE	Platform 1.0.9 · Android versionCode 9 · build 3730b06b
DOCUMENT DATE	9 September 2026

1,636 automated tests

3 shipped clients

10 decision engines

17 production tables

46 API endpoints

Every version number, table name, endpoint, coefficient and threshold in this document was read out of the working tree on the date above. This is an engineering description of a running system, not a design proposal.

Contents

Twelve sections — the stack top to bottom, then one trip end to end

1	What the platform does	3	7	Layer 4 — Data platform	15
2	System architecture	5	8	Layer 5 — Accessibility intelligence	17
3	How one trip flows, end to end	7	9	Layer 6 — External data providers	19
4	Layer 1 — Driver mobile client	9	10	Security architecture	20
5	Layer 2 — Manager web client	11	11	Testing and quality	22
6	Layer 3 — API and domain services	12	12	Build, release and deployment	23

The system in five numbers

1,636 automated tests passing	3 shipped clients web · Android · API	10 deterministic decision engines	17 production database tables	71 MB installable Android APK
-------------------------------------	---	---	-------------------------------------	-------------------------------------

THE ONE RULE THAT SHAPES EVERY LAYER BELOW

Roads are treated as changeably passable, not as fixed lines on a map. A corridor that was open this morning may be closed by a landslide this afternoon and open again tomorrow. Every layer of this stack exists to carry that fact accurately: the database stores road condition as dated evidence rather than as a property, the engines refuse a corridor rather than scoring it low, and the clients display a missing input as missing rather than as a safe one.

1 What the platform does

The operating problem in the North East, and the six capabilities that answer it

Freight into and across the eight North Eastern states passes through a 22 km chokepoint at Siliguri, over single corridors whose next alternative is often more than 100 km away, through a monsoon that closes highways from May to September, and across cellular dead zones where a truck is simply unreachable for hours. A fleet manager's best instrument today is a phone call, and fuel is budgeted by a rule of thumb that models neither gradient nor payload.

The six capabilities

Accessibility intelligence

Weather, landslide exposure and closure evidence are combined per corridor and re-evaluated live, so the question is never "what is the shortest road" but "which of these roads is usable right now, and on what evidence".

Three-corridor explainable comparison

Primary, fuel-saving and backup corridors compared side by side on distance, ETA, litres, CO₂ and reason-coded risk — with the deltas stated in points, minutes and kilometres rather than as a percentage of an ordinal score.

Physics-based fuel and emissions

Litres and CO₂ derived from payload mass, cumulative climb metres and speed through a CMEM-form model with published coefficients — not from distance multiplied by a fleet average.

Fleet Sentinel escalation

A truck that has not moved beyond a PostGIS radius for a set interval and whose driver has not answered produces a populated SOS briefing to the manager automatically. The escalation is a rule, so it fires whether or not anyone was watching.

Offline-first driver client

Route geometry, turn maneuvers, roadside POIs and the emergency guide are cached before the dead zone; GPS fixes queue locally and replay idempotently on `(trip_id, device_fix_id)` when signal returns.

Vernacular in-cab assistance

Reason codes — not sentences — cross the wire, so Assamese, Bengali, Hindi and English guidance is produced by lookup rather than translation, and works with no model and no network.

Four design commitments that a judge can check in the code

Commitment	What it means in the implementation
Silence is not evidence	No amount of elapsed time may move a road segment toward <code>VERIFIED_OPEN</code> . A repair claim is a distinct state (<code>REPAIR_REPORTED</code>) from a confirmation. Enforced in <code>domain/road_memory.py</code> .
A closed road is refused, not penalised	<code>route_eligibility</code> runs <i>before</i> ranking and returns <code>REJECTED</code> . A penalty can be out-voted by other components; a refusal cannot.
UNKNOWN ≠ SAFE, STALE ≠ LIVE	A factor with no usable reading is reported as <code>NOT_AVAILABLE</code> with a reason code. Stale observations are counted and named, never scored. A provider outage can never become <code>risk = 0</code> .
AI is never in the safety path	Every engine is a deterministic weighted rule with published constants. A test asserts that no <code>confidence</code> , <code>probability</code> or <code>model_version</code> field ever reaches the wire. The language model only rewords facts the rules already produced.

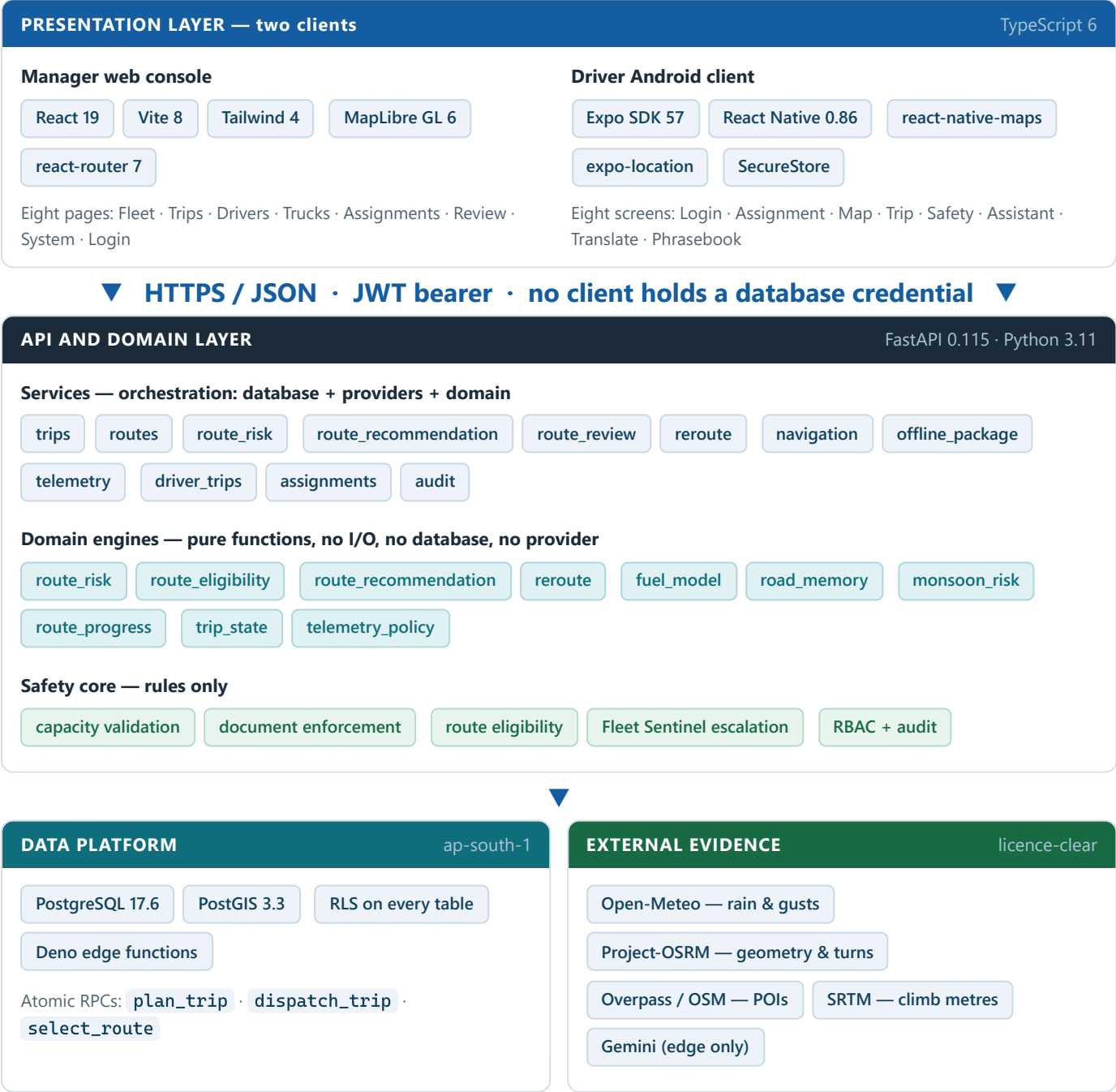
WHY DETERMINISTIC RULES ARE THE ENGINEERING ANSWER HERE, NOT A COMPROMISE

After a real incident, "*the truck had not moved more than 50 m in 60 minutes and the driver did not answer in 30*" is a defensible sentence in an inquiry. "*The model scored 0.83*" is not. Every constant in this system is published, versioned and arguable in review — and the append-only evidence log the platform produces is precisely the dataset that any future statistical model would have to beat.

2 System architecture

Two purpose-built clients, one API, one data platform, one intelligence plane

The manager works at a desk with several screens and a keyboard. The driver works one-handed in a cab, often at night, frequently with no signal. Those are genuinely different design constraints, so they are two applications rather than one responsive web app that would compromise both. They share one backend, one schema and one set of rules.



The architectural rule that makes this testable

APP/DOMAIN/ IMPORTS NOTHING

`app/domain/` contains no import of SQLAlchemy, httpx or FastAPI. Every engine is a pure function over dataclasses. That is why risk, fuel, eligibility and reroute logic can be exercised exhaustively without a database or a network, why the constants can be argued with in review, and why the same logic is guaranteed identical whether it was reached from the manager console, the Android client or a direct API call.

Why clients never touch the database directly

Capacity rules, document enforcement and the safety path are only unbypassable if there is exactly one way to write. No client holds a database credential; Row Level Security is enabled on every table; and operations that must be atomic across several writes are server-side RPCs rather than a sequence of client calls that a dropped connection can leave half-applied.

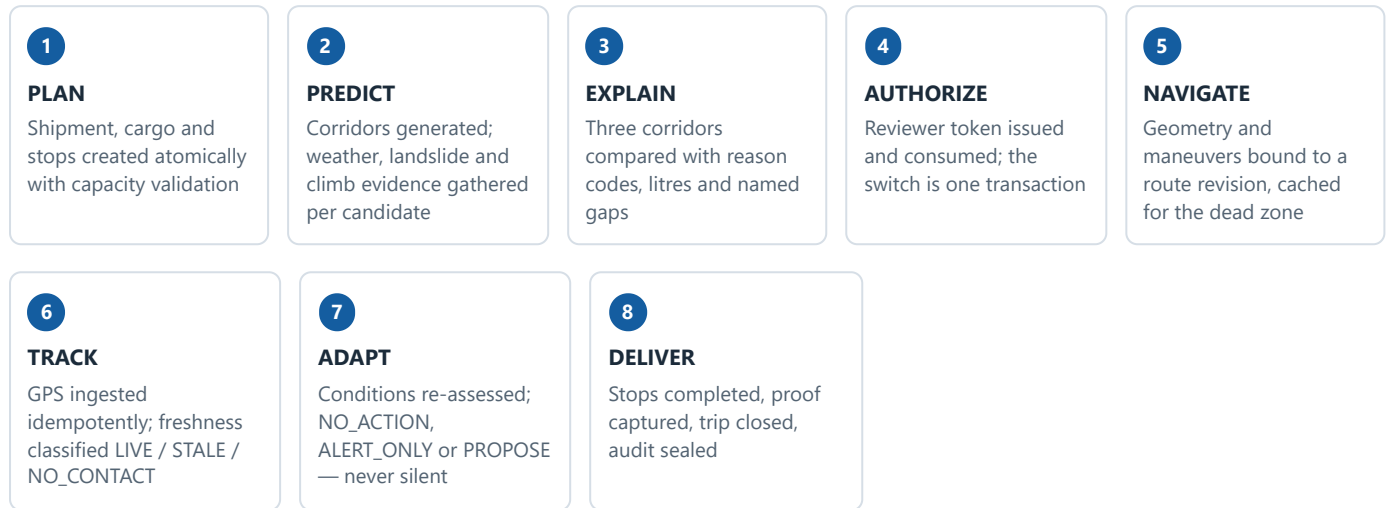
Version inventory

Layer	Core technology	Pinned version
Driver client	Expo · React Native	<code>expo ~57.0.18</code> · <code>react-native 0.86.3</code> · <code>react 19.2.3</code>
Manager client	React · Vite · Tailwind	<code>react ^19.2.8</code> · <code>vite ^8.2.2</code> · <code>tailwindcss ^4.3.3</code>
API	FastAPI · Uvicorn	<code>fastapi 0.115.6</code> · <code>uvicorn[standard] 0.34.0</code>
Backend language	Python	3.11.9 (<code>>=3.11,<3.13</code> in pyproject)
ORM / migrations	SQLAlchemy · Alembic	<code>sqlalchemy 2.0.36</code> · <code>alembic 1.14.0</code>
Database driver	psycopg 3 (binary)	<code>psycopg[binary] 3.2.3</code>
Geospatial ORM	GeoAlchemy2	<code>geoalchemy2 0.16.0</code>
Database	PostgreSQL + PostGIS	17.6 · PostGIS 3.3 · region ap-south-1
Validation	Pydantic	<code>pydantic 2.10.4</code> · <code>pydantic-settings 2.7.0</code>
Auth	Argon2id · PyJWT	<code>argon2-cffi 23.1.0</code> · <code>pyjwt[crypto] 2.10.1</code>
Maps	MapLibre · Leaflet · RN Maps	<code>maplibre-gl ^6.6.0</code> · <code>leaflet ^1.9.4</code> · <code>react-native-maps 1.27.2</code>
Tests	pytest · Vitest	<code>pytest 8.3.4</code> · <code>vitest ^3.2.7 / ^4.1.11</code>

3 How one trip flows, end to end

Eight stages, from the manager's plan to proof of delivery — with the real call at each step

This is the spine of the product. Everything in sections 4 to 9 exists to serve one of these eight stages. The scenario: a 14-tonne consignment from Guwahati to Shillong during monsoon.



Stage by stage, with the actual call

1	Manager Trips page	Enters pickup, destination and cargo. <code>POST /api/trips/plan</code> → Supabase RPC <code>plan_trip</code> creates shipment, cargo items, trip and stops in one transaction , validating that total cargo mass does not exceed the assigned truck's rated capacity. Over-capacity is refused here, before dispatch — not discovered at the weighbridge.
2	Platform routing + evidence	OSRM returns three candidate corridors with geometry, duration and turn maneuvers. For each, Open-Meteo is sampled at points <i>along the polyline</i> (not just at the endpoints), SRTM supplies cumulative climb metres, and landslide exposure is looked up. Each reading carries a freshness stamp. Stored as <code>trip_routes</code> rows with <code>geography(LineString,4326)</code> geometry.
3	Platform domain engines	<code>route_eligibility</code> runs first — a confirmed closure returns <code>REJECTED</code> and the corridor leaves the comparison entirely. <code>route_risk</code> scores the survivors; <code>fuel_model</code> computes litres and CO ₂ ; <code>route_recommendation</code> applies an evidence-parity gate and refuses to compare two corridors that were scored from different available inputs. <code>GET /api/trips/{id}/routes/recommendation</code>
4	Manager Route review	Sees three corridors with distance, ETA, litres, CO ₂ , a risk band, the reason codes behind it, and — explicitly — any factor that could not be obtained. Selecting a corridor that requires review consumes a single-use 30-minute token bound to a digest of the exact evidence shown. <code>POST /api/trips/{id}/routes/{rid}/select</code> → RPC <code>select_route</code> : demote, promote and re-point the trip in one transaction .
5	Driver Android client	Accepts the dispatched trip, verifies the truck (photo, registration, odometer, fuel level, damage notes), then downloads the navigation and offline packages — <code>GET /api/driver/me/trip/navigation</code> and <code>.../offline-package</code> — carrying geometry, maneuvers, stops, POIs and the emergency guide. Both are bound to a route revision , so a reroute landing mid-request can never draw a new line with old turns.
6	Driver → Platform telemetry	<code>POST /api/driver/me/location</code> streams GPS fixes, batched and de-duplicated on device, idempotent on (trip_id, device_fix_id) so a replayed offline queue cannot double-count. <code>telemetry_policy</code> classifies each truck LIVE / STALE / NO_CONTACT by fix age; <code>route_progress</code> computes progress along the selected corridor.

- 7 **Fleet Sentinel**
safety core
- A PostGIS `ST_DWithin` check establishes that the vehicle has not left a radius for the configured interval. Stationary *and* unanswered escalates to a populated SOS briefing — last known position, cargo, driver contact, elapsed silence — delivered to the manager by rule rather than by anyone happening to look.
-
- 8 **Platform → Manager**
reroute
- `GET /api/trips/{id}/reroute` returns exactly one of `NO_ACTION`, `ALERT_ONLY` or `PROPOSE`. `ALERT_ONLY` exists because most of the North East is a single corridor: "*this road has deteriorated and there is nothing better*" is real, actionable information. A proposal requires human acceptance — `POST .../reroute/accept` — so the driver's map never changes underneath them.
-
- 9 **Driver**
delivery
- Arrive and complete each stop, then complete the trip. Every transition is checked against `trip_state`'s legal-transition table, written to the append-only `trip_events` log with an optional location, and mirrored into `audit_logs` with actor, before/after JSON, reason and IP.

WHAT MAKES THIS FLOW SURVIVE THE NORTH EAST SPECIFICALLY

Three properties, none of them cosmetic. **Atomicity** — plan, dispatch and route switch are single transactions, so a dropped connection cannot leave the manager looking at corridor B while the driver is authoritatively on corridor A. **Idempotency** — a replayed GPS queue produces the same state as a live stream. **Revision binding** — geometry and maneuvers are stamped with the route revision they belong to, and a client discards a package whose route no longer matches.

4 Layer 1 — Driver mobile client

Expo SDK 57 · React Native 0.86 · one Android build, phone-only field kit

A phone-only field client. One hand, poor signal, sometimes at night, sometimes with no network at all. Every design constraint below follows from that — including the release gate that makes a laptop-dependent build impossible to ship.

Screens

Screen	What it does
MapScreen	The cockpit — corridor polyline, next-turn banner with distance countdown, live speed, POI discovery, recentre / fit / mute controls
TripScreen	Stop list, arrive and complete actions, cargo detail, trip progress against the planned ETA
AssignmentScreen	Truck verification — photo, registration, odometer, fuel level, damage notes; mismatches flagged for manager review
SafetyScreen	Offline emergency guide, 112 / 108 / 1033 dial shortcuts, 45-minute rest tracker
AssistantScreen	Trip-scoped question answering in four sub-modes, backed by the edge AI function
PhrasebookScreen	Reviewed offline phrase pack — works with no model, no server and no internet
TranslateBox	Machine translation of one short message, labelled on screen as machine output
LoginScreen	Email and password; token written to the hardware-backed keystore

Device capabilities and why each is required

Package	Version	Why it is required
<code>expo-location</code>	~57.0.14	Foreground GPS streaming — the single input every live feature depends on
<code>expo-secure-store</code>	~57.0.2	Hardware-backed keystore for the refresh token; tokens never touch AsyncStorage
<code>@react-native-async-storage</code>	2.2.0	Offline queue for GPS fixes and the cached route package
<code>expo-speech</code>	~57.0.2	Spoken turn guidance — a driver on a hill road should not be reading a screen
<code>react-native-maps</code>	1.27.2	Native Android map surface, route polyline, live vehicle marker
<code>leaflet</code>	^1.9.4	Web build of the same screen, so the map is testable in CI without a device farm
<code>react-native-safe-area-context</code>	~5.7.0	Notch- and gesture-bar-safe layout for the map cockpit

ONE IMPORT, TWO RENDERING SURFACES

The map component is split by platform extension — `DriverRouteMap.native.tsx` and `DriverRouteMap.web.tsx` — behind a single import. Screen logic is written once; only the rendering surface differs. That is what allows the entire driver UI to be exercised by 510 Vitest tests over `react-native-web` in jsdom, with no emulator in the loop.

Supporting modules

Module	Responsibility
<code>src/tracking/tracker.ts</code>	GPS acquisition, batching and de-duplication before upload
<code>src/tracking/useLocationTracking.ts</code>	Permission lifecycle, foreground subscription, revocation handling

Module	Responsibility
<code>src/auth/tokenStore.ts</code>	SecureStore-backed token persistence and refresh rotation
<code>src/trip/TripProvider.tsx</code>	Server trip state synchronisation and offline-state transitions
<code>src/map/useRouteGeometry.ts</code>	Package fetch, cached fallback for the same trip <i>and</i> route, honest error state otherwise
<code>scripts/check-release-config.mjs</code>	Release gate — fails the EAS build outright if a LAN backend URL is configured

THE RELEASE GATE — THE CONTROL THAT ACTUALLY GUARANTEES A FIELD-USABLE APK

`EXPO_PUBLIC_API_BASE_URL` means "the laptop on the LAN". The build gate **refuses to build** when it is set, so a laptop-dependent APK cannot ship. The hosted intelligence origin has its own variable, `EXPO_PUBLIC_INTELLIGENCE_BASE_URL`, held to a stricter rule: a private network address, a plain-http origin or a loopback host is rejected at build time. The rule lives once, in `releaseConfig.mjs`, shared by the gate and the runtime — because a rule restated in two places is a rule that drifts.

Offline behaviour, precisely

Condition	Behaviour
Network lost mid-trip	Cached package continues to drive the map and turn banner; source is labelled <code>CACHED</code> with its capture time, never presented as <code>LIVE</code>
GPS fixes with no network	Queued locally, replayed idempotently on <code>(trip_id, device_fix_id)</code> when signal returns
Reroute lands mid-request	<code>matches()</code> discards geometry whose route id changed — old turns are never drawn on a new line
Location permission revoked	Tracking stops, the manager sees <code>NO_CONTACT</code> , and the screen says so rather than freezing the last position
Emergency with no signal	Safety guide, rest tracker and 112 / 108 / 1033 shortcuts are fully offline

5 Layer 2 — Manager web client

React 19 · Vite 8 · Tailwind 4 · MapLibre GL — a dense GIS console, not a dashboard

Pages

- **FleetPage** — live map, KPI bar, truck context drawer
- **TripsPage** — atomic plan, dispatch, cancel, close
- **DriversPage** — roster, documents, deactivation
- **TrucksPage** — fleet register, capacity, retirement
- **AssignmentsPage** — driver ↔ truck pairing and verification
- **ReviewPage** — authorised route review queue
- **SystemPage** — backend health and provider status
- **LoginPage** — role-aware landing

Key components

- **FleetMap** — MapLibre canvas; a test asserts the production build actually emits the worker
- **FleetKpiBar** — Active / Moving / Stale / No-contact
- **TruckContextDrawer** — cargo utilisation, live speed, call driver
- **RouteRiskComparison** — the three-corridor stepper with the fuel bar chart
- **TripRouteReview** — authorisation modal and review states
- **AddressPicker** — geocode search with a map-pick alternative
- **track.ts** — observed GPS breadcrumb against the planned corridor

The two views a manager actually works from

View	What it shows, and why it is built that way
Fleet map	Planned corridor and observed breadcrumb are drawn as <i>two separate lines</i> , so divergence is visible rather than inferred. Marker state is driven by <code>telemetry_policy</code> , so "stale" on the map and "stale" in the API are the same computation, not two implementations of the same word.
Route comparison	Three corridors side by side. Deltas are stated in points, minutes and kilometres — never as a percentage, because a risk score is an ordinal on this project's own scale and a percentage of an ordinal is not a meaningful quantity. Any factor that could not be obtained is named on screen next to the score it is missing from.

Package inventory

Package	Version	Role
<code>react</code> / <code>react-dom</code>	<code>^19.2.8</code>	UI runtime, shared version line with the driver client
<code>react-router-dom</code>	<code>^7.18.3</code>	Routing across the eight manager pages
<code>vite</code>	<code>^8.2.2</code>	Dev server and production bundler
<code>tailwindcss</code>	<code>^4.3.3</code>	Utility styling; the GIS console theme is expressed as design tokens
<code>maplibre-gl</code>	<code>^6.6.0</code>	Vector tiles, worker-based rendering, licence-free
<code>@supabase/supabase-js</code>	<code>^2.116.0</code>	Authenticated read path and RPC calls
<code>oxlint</code>	<code>^1.79.0</code>	Rust-based linting, chosen for speed on a large surface
<code>vitest</code> + Testing Library	<code>^3.2.7</code>	150 component and page tests in jsdom

6 Layer 3 — API and domain services

FastAPI 0.115 · Python 3.11 · 46 endpoints · pure domain core

Application structure

```
backend/app/
├─ main.py          app assembly, middleware, router mounting
├─ api/             HTTP surface
│  ├─ auth.py       login · refresh · logout · me
│  ├─ fleet.py       drivers · trucks · assignments
│  ├─ trips.py       shipments · trips · routes · risk · reroute · fleet/active
│  ├─ driver.py      the driver's own trip, GPS ingest, offline + nav packages
│  ├─ geocoding.py   address suggest / details
│  ├─ ai.py          AI availability and question answering
│  ├─ health.py      /health (liveness) · /ready (dependency-checked)
│  └─ deps.py        auth + permission dependencies
├─ domain/          PURE LOGIC — no I/O, no DB, no provider
│  ├─ route_risk.py  weighted risk rule, published constants
│  ├─ route_eligibility.py may this route be taken at all
│  ├─ route_recommendation.py explainable comparison + evidence parity
│  ├─ reroute.py     NO_ACTION / ALERT_ONLY / PROPOSE
│  ├─ fuel_model.py  CMEM-form physics fuel + CO2
│  ├─ road_memory.py 5-state road evidence machine
│  ├─ monsoon_risk.py recurrence · rainfall · season · status
│  ├─ route_progress.py progress along a corridor from GPS
│  ├─ trip_state.py  legal trip state transitions
│  ├─ telemetry_policy.py LIVE / STALE / NO_CONTACT thresholds
│  ├─ weather.py     observation model + freshness
│  ├─ landslide.py   exposure vs current hazard
│  ├─ places.py       POI categories and ranking
│  ├─ routing.py      provider-agnostic route model
│  └─ ai_prompts.py  prompt templates and refusal rules
├─ services/        orchestration: DB + providers + domain
├─ models/          SQLAlchemy tables and enums
├─ schemas/         Pydantic request/response contracts
├─ core/            config · security · permissions · rate limit · errors
├─ db/              engine, session, provider selection
└─ auth/            token verification
```

Endpoint surface

Authentication and identity

Method	Path	Purpose
GET	/health	Liveness — checks no dependency, always 200 while the process is up
GET	/ready	Readiness — real PostgreSQL and PostGIS version strings, 503 when unhealthy
POST	/api/auth/login	Argon2id verification, access + refresh issue, rate limited per IP and per identifier
POST	/api/auth/refresh	Rotating refresh with reuse detection and family revocation
POST	/api/auth/logout	Revoke the presented refresh token
GET	/api/auth/me	Current principal, role and permissions

Fleet administration

Method	Path	Purpose
GET / POST	/api/drivers	List and create drivers
GET / PATCH	/api/drivers/{id}	Read and update one driver
POST	/api/drivers/{id}/deactivate	Soft-deactivate, audited
GET / POST	/api/trucks	List and create trucks
GET / PATCH	/api/trucks/{id}	Read and update one truck
POST	/api/trucks/{id}/retire	Retire a truck, audited
GET / POST	/api/assignments	List and create driver ↔ truck assignments
POST	/api/assignments/{id}/verify	Manager review of the driver's verification submission
POST	/api/assignments/{id}/end	End an active assignment

Trips, corridors and accessibility

Method	Path	Purpose
POST	/api/trips/plan	Atomic shipment + trip + stops creation with capacity validation
GET	/api/trips/{id}	Full trip detail with stops, routes and events
POST	/api/trips/{id}/dispatch	Dispatch to the driver
POST	/api/trips/{id}/cancel · /close	Cancel with reason, or close a delivered trip — both audited
GET	/api/trips/{id}/routes	The candidate corridors for this trip
POST	/api/trips/{id}/routes/recalculate	Re-run the routing provider and re-score
GET	/api/trips/{id}/routes/{rid}/risk	Full assessment with components, reason codes and named gaps
GET	/api/trips/{id}/routes/recommendation	Explainable comparison across candidates
GET/POST/DELETE	.../review-authorization	Issue, read and revoke a single-use 30-minute reviewer token
POST	/api/trips/{id}/routes/{rid}/select	Select a corridor, consuming the authorisation when one is required
GET	/api/trips/{id}/reroute	Assessment: NO_ACTION · ALERT_ONLY · PROPOSE
POST	/api/trips/{id}/reroute/accept	Human acceptance of a proposed corridor change
GET	/api/trips/{id}/track	Observed GPS breadcrumb for the trip
GET	/api/fleet/active	Live fleet snapshot with freshness classification

Driver-scoped surface

Method	Path	Purpose
GET	/api/driver/me · /assignment	The signed-in driver's profile and current assignment state
POST	/api/driver/me/assignment/verify	Submit truck verification evidence
GET	/api/driver/me/trip	The driver's current trip
GET	/api/driver/me/trip/navigation	Route geometry + turn maneuvers, bound to a route revision

Method	Path	Purpose
GET	/api/driver/me/trip/offline-package	Everything needed to keep working with no network
GET	/api/driver/me/trip/places	Roadside POIs: near me · along route · in viewport
POST	/api/driver/me/trip/accept · /start · /complete	Trip lifecycle transitions
POST	.../trip/stops/{id}/arrive · /complete	Stop-level progress
POST	/api/driver/me/location	GPS ingest — idempotent on (trip_id, device_fix_id)
GET	/api/geocoding/suggest · /details	Address suggestions and coordinate resolution
GET / POST	/api/ai/status · /api/ai/ask	Model reachability, and trip-scoped bounded question answering

7 Layer 4 — Data platform

PostgreSQL 17.6 + PostGIS 3.3 in ap-south-1 — corridor geometry as a first-class type

Component	Detail
Primary database	Supabase-managed PostgreSQL 17.6, region ap-south-1 (Mumbai) — data stays in India, which matters for a government-sponsored logistics dataset
Geospatial extension	PostGIS 3.3 — <code>geography(Point,4326)</code> and <code>geography(LineString,4326)</code>
Connection mode	Session pooler on port 5432 — <i>not</i> the transaction pooler on 6543, which breaks psycopg prepared statements and is unsafe for Alembic DDL
Row Level Security	Enabled on every table, because Supabase publishes <code>public</code> through its Data API
Atomic RPCs	<code>plan_trip</code> · <code>dispatch_trip</code> · <code>select_route</code> — the three operations that must not be half-applied
Edge Functions (Deno)	<code>gemini-ai</code> · <code>driver-trip</code> · <code>resolve-map-link</code>
Migrations	9 Alembic revisions; Alembic follows <code>DATABASE_PROVIDER</code> , so it can never migrate a different database than the one being served

Schema — 17 production tables

Table	Cols	Notes
<code>users</code>	9	Argon2id hash, role, activity flag
<code>drivers</code>	16	Licence, emergency contact, salary (permission-gated)
<code>driver_documents</code>	12	Type, expiry, verification actor and timestamp
<code>trucks</code>	18	Capacity, axles, dimensions, baseline mileage, odometer
<code>truck_documents</code>	12	RC, insurance, fitness, PUC, permits
<code>truck_maintenance</code>	10	Service / repair / breakdown / inspection history
<code>driver_truck_assignments</code>	15	Verification photo, reported registration / odometer / fuel, mismatch flag
<code>shipments</code>	15	PostGIS pickup and destination points, priority, status
<code>cargo_items</code>	10	Weight, quantity, hazardous and perishable flags
<code>trips</code>	21	Selected route, planned vs current ETA, delay minutes, lifecycle timestamps
<code>trip_stops</code>	14	Sequence, kind, geofence radius, planned vs actual arrival
<code>trip_routes</code>	18	LineString geometry, distance, duration, fuel, risk score and factors, maneuvers, provider id
<code>trip_events</code>	9	Append-only lifecycle log with optional location and JSON payload
<code>gps_points</code>	13	Point geography, speed, heading, accuracy, <code>device_fix_id</code> for idempotency, mock-location flag
<code>refresh_tokens</code>	11	Family id, replacement chain, revocation reason, user agent, IP
<code>audit_logs</code>	10	Actor, action, entity, before / after JSON, reason, IP
<code>route_review_authorizations</code>	18	Evidence digest and snapshot, policy and evidence versions, issue / consume / revoke trail

Migration history

Revision	What it introduced
0001_bootstrap_postgis	PostGIS extension and spatial groundwork
0002_core_domain	Users, drivers, trucks, documents, assignments, shipments, trips, stops, routes, events, GPS points
0003_refresh_tokens	Rotating refresh-token family table
0004_audit_actor_restrict	Audit actor FK made restrictive so history cannot be orphaned
0005_trip_event_location_null	Trip events may legitimately have no location
0006_current_assignment_unique	Partial unique index — one active assignment per driver
0007_route_review_authorizations	Single-use reviewer tokens with evidence digest and expiry
0008_trip_driver_acceptance	Explicit driver acceptance timestamps
0009_route_maneuvers	Turn-by-turn maneuver storage bound to a route revision

WHY THE ROUTE SWITCH IS A DATABASE FUNCTION, NOT THREE CLIENT WRITES

Selecting a corridor means three changes: demote the previously selected route, promote the new one, and re-point `trips.selected_route_id`. Performed as independent writes from a browser, each can fail or be lost separately — producing zero selected routes, two selected routes, or a trip whose pointer disagrees with its own route table. On a platform whose entire subject is weak connectivity, a half-applied route change is the one outcome that must be impossible. `select_route` makes it one transaction, with the currently-selected route as the concurrency token so two managers acting at once cannot both succeed.

8 Layer 5 — Accessibility intelligence

Ten deterministic engines with published constants — the core of the submission

THE RULE THIS WHOLE LAYER IS BUILT AROUND

Every engine below is a **deterministic weighted rule with published constants**. Each carries a version name that says so, and a test asserts that no confidence, probability or model-version field ever reaches the wire. The language model is a separate thing entirely: it rewords facts the engines have already produced, and it has no write path to any decision.

The five-stage accessibility pipeline

1 ROUTE OSRM returns geometry, duration and maneuvers for each candidate corridor	2 WEATHER Open-Meteo sampled at points along the polyline; every reading carries a freshness stamp	3 LANDSLIDE Exposure to known zones, kept distinct from a current hazard report	4 FUEL Physics over distance, payload, climb and speed → litres and CO ₂	5 SYNTHESIS Weighted score, band, reason codes, component breakdown and named gaps
---	--	---	---	--

The engines

Engine	Kind	Method and published constants
<code>route_risk</code>	Weighted rule	Rain ≤ 45 pts · wind ≤ 25 · duration ≤ 20 · distance ≤ 10, plus landslide points. Bands: LOW < 30 ≤ MODERATE < 60 ≤ HIGH . Thresholds: rain moderate 2.5 mm, heavy 7.5 mm; gust notable 40 km/h, severe 60 km/h. Stale observations are counted and reported, never scored.
<code>route_eligibility</code>	Refusal gate	Runs before ranking. A confirmed closure is REJECTED , not penalised — a penalty can be out-voted by other components, a refusal cannot. Unknown landslide evidence forces REQUIRES_REVIEW .
<code>fuel_model</code>	Physics (CMEM / SAE J1321)	$F = (B + K_{\text{payload}} \cdot M + K_{\text{speed}} \cdot \max(0, v - 50)) \cdot D + K_{\text{grade}} \cdot \max(0, \Delta H)$ with $B = 0.22 \text{ L/km}$, $K_{\text{payload}} = 0.0075 \text{ L/(t·km)}$, $K_{\text{grade}} = 0.00018 \text{ L/m}$, $K_{\text{speed}} = 0.002 \cdot \text{CO}_2 \text{ at } 2.68 \text{ kg/L}$.
<code>road_memory</code>	Evidence state machine	Five states: VERIFIED_OPEN · REPORTED_INCIDENT · CLOSED · REPAIR_REPORTED · AWAITING_VERIFICATION . Silence is not evidence — no elapsed time may move a road toward VERIFIED_OPEN . Evidence carries a source; FLEET_TRAVERSAL is the strongest.
<code>monsoon_risk</code>	Weighted rule	Recurrence · rainfall · season · last observed road status. Passability is a separate three-valued field, not a score, so a closed segment cannot narrowly lose a comparison.
<code>route_recommendation</code>	Comparison rule	Deltas in points, minutes and kilometres — never as a percentage, because a risk score is an ordinal on this project's own scale. Refuses to compare two routes scored from different available inputs.
<code>reroute</code>	Threshold + comparison	Three outcomes: NO_ACTION · ALERT_ONLY · PROPOSE . ALERT_ONLY exists because most of the North East is a single corridor and "this road has deteriorated and there is nothing better" is real information.
<code>telemetry_policy</code>	Freshness rule	LIVE / STALE / NO_CONTACT from fix age; drives the fleet KPI bar and every map marker state.
<code>route_progress</code>	Geometric	Progress along the selected corridor computed from the observed GPS track, so a delay is measured rather than assumed.
<code>trip_state</code>	Transition table	The legal state graph for a trip. An illegal transition is refused at the domain layer, not caught by a UI that happens to hide the button.

Evidence integrity — how the risk response is shaped

A `RouteRisk` never returns a bare number. It carries `inputs` (each factor marked `AVAILABLE` or `NOT_AVAILABLE`), `unavailable` , `reason_codes` , `observations_used` and `observations_stale` alongside `score` — and the manager UI renders the gap next to the score it is missing from. The reason-code catalogue carries the exact semantic the problem statement requires:

REASON CODE — WEATHER_UNAVAILABLE

"Hazard data for this route is unavailable. This is not a clear route — it is an unchecked one."

When no plane is reachable, candidates return `score: null` , `band: 'UNASSESSED'` , `eligibility: 'NOT_ASSESSED'` . **Null rather than zero is the entire point:** zero is a legitimate assessed value meaning "we looked and found nothing wrong". Null means nobody looked. The two must never render identically.

Reason codes, not sentences — how vernacular support actually works

The API sends `HEAVY_RAIN_ON_CORRIDOR` , not "heavy rain on the corridor". The client resolves it through `i18n/reasonCodes.json` . That single decision is what makes Assamese, Bengali, Hindi and English in-cab guidance a lookup rather than a translation call — so it is instant, identical across clients, reviewable by a human translator, and works with no model and no network.

The language model surface

Element	Detail
Primary model	Google Gemini Flash via the <code>gemini-ai</code> Supabase Edge Function; default <code>gemini-3-flash-preview</code> , configurable through <code>GEMINI_MODEL</code>
Fallback	OpenRouter, default <code>google/gemma-4-31b-it:free</code> , 15-second budget
Key handling	<code>GEMINI_API_KEY</code> and <code>OPENROUTER_API_KEY</code> live server-side in the edge function. No client holds either.
Status model	One <code>buildStatus()</code> for both routes: <code>OFFLINE</code> · <code>CONFIGURED</code> · <code>USABLE</code> · <code>QUOTA_EXHAUSTED</code> · <code>FALLBACK</code>
Defences	Adversarial prompt-injection guards, chain-of-thought stripping (<code>cleanAiText</code>), per-minute rate limiting
Hard boundaries	Driver-scoped — every route takes <code>CurrentDriver</code> with no id parameter. One concurrent generation. Prompt-length and output-token caps. Stateless: no conversation is stored.
Forbidden outputs	Medical advice, medicine names or doses; a road-is-clear verdict; a promise that help is coming; turning an unavailable fact into a value. Each pinned by a test.

9 Layer 6 — External data providers

All licence-clear and free of per-call billing — and every failure mode is defined

Provider	Endpoint	Used for	Behaviour on failure
Open-Meteo	<code>api.open-meteo.com</code>	Rainfall intensity and wind gusts along the corridor	The request still succeeds; the factor is marked <code>NOT_AVAILABLE</code> with a reason code. A 503 here must not hide distance and duration evidence that is still perfectly good.
Project-OSRM	<code>router.project-osrm.org</code>	Road geometry, duration, turn maneuvers	Configurable primary URL with this as fallback; 8-second timeout
Overpass / OSM	<code>overpass-api.de</code>	Roadside POIs — fuel, tyre repair, hospitals, rest stops	Snapshot service degrades to cached results
NASA SRTM	elevation dataset	Cumulative climb metres for the fuel model	No elevation → the fuel model returns its baseline or null, never an imputed value
Google Gemini	via Supabase Edge Function	Vernacular assistant, safety rewording, translation	Quota exhausted → OpenRouter fallback → otherwise every AI surface renders its unavailable state
OpenRouter	<code>openrouter.ai/api/v1</code>	Backup generation engine	15-second budget, then unavailable

THE PROVIDER CONTRACT, IN ONE LINE

A provider failure is displayed as a provider failure. It is never converted into a number. `UNKNOWN` ≠ `SAFE` and `STALE` ≠ `LIVE` are enforced in the domain layer, not left to the UI to remember.

Hazard and closure evidence

Landslide hazard, flood and damage reporting, and road closure bulletins enter the platform through `road_memory` — the five-state evidence machine — rather than through a score. Each item of evidence carries its **source**, its **date** and its **state**, and the strongest source is a `FLEET_TRAVERSAL`: one of our own trucks having driven the segment.

Source	Contributes
Geological Survey of India · <code>bhukosh.gsi.gov.in</code>	Landslide inventory and hazard exposure per zone
ASDMA · <code>asdma.gov.in</code>	Flood and damage reports
IMD · <code>mausam.imd.gov.in</code>	District rainfall warnings
NHAI / BRO / state PWD	Closure and reopening bulletins, held per segment as dated evidence
Fleet traversal <code>STRONGEST</code>	A GPS track across the segment — the one source that can reopen a road when no bulletin carries it

OUR OWN RESEARCH FINDING, AND THE DESIGN IT PRODUCED

No Indian source publishes road *reopening*. GSI forecasts hazard, ASDMA reports damage, IMD reports rain — but reopening is decided by PWD, NHAI or BRO and communicated ad hoc. That single finding is why our own fleet traversals are treated as the strongest evidence a segment is open, and why the road-evidence log is append-only and per segment: over a season of operation, the platform accumulates the passability record for the region that no source currently holds.

10 Security architecture

Identity, authorisation, audit and the structural guards that make misconfiguration fail loudly

Control	Technology	Detail
Password hashing	<code>argon2-cffi 23.1.0</code>	Argon2id (RFC 9106)
Tokens	<code>pyjwt[crypto] 2.10.1</code>	15-minute access token, 30-day refresh
Refresh rotation	Token families	Every refresh issues a new token and revokes the old one. Reuse of a revoked token revokes the whole family — the standard detection for a stolen refresh token.
Federated verification	Supabase JWKS	ES256/RS256 verified against the project JWKS. Algorithm pinned , never read from the token, so <code>alg: none</code> and HS256-signed-with-the-public-key are both refused. <code>iss</code> , <code>aud</code> , <code>exp</code> , <code>iat</code> and <code>sub</code> all required.
Rate limiting	In-process limiter	Login 20/min per IP and 10/min per identifier; refresh 60/min per IP; 60-second window
Authorisation	Permission constants	Four roles — <code>ADMIN</code> · <code>MANAGER</code> · <code>DRIVER</code> · <code>AUTHORISED_REVIEWER</code> — against ~28 named permissions such as <code>route:review_authorize</code> , <code>driver:read_sensitive</code> , <code>trip:execute_own</code> , <code>location:submit_own</code>
Audit	<code>audit_logs</code>	Append-only: actor, action, entity, before / after JSON, reason, IP
Route authorisation	<code>route_review_authorizations</code>	Single-use, 30-minute, cryptographically bound to an evidence digest and a policy version
Database isolation	PostgreSQL RLS	Enabled on every table; no client holds a database credential
Transport	TLS · <code>DB_REQUIRE_SSL</code>	SSL required to the database by default

Roles are read from the database, never from the token

WHY THIS MATTERS MORE THAN IT SOUNDS

A federated identity provider says `authenticated` for everyone. `deps.get_current_user` therefore reads the real role from the `users` table on **every request**. The consequence is operational, not theoretical: a demotion or a deactivation takes effect immediately, rather than whenever the user's token happens to expire. A revoked reviewer cannot authorise a corridor with a token issued three minutes ago.

Structural guards — misconfiguration fails loudly rather than quietly

Guard	Behaviour
Secret hygiene	The backend refuses to start with the placeholder <code>SECRET_KEY</code> when <code>APP_ENV</code> is not development. <code>VITE_*</code> and <code>EXPO_PUBLIC_*</code> are inlined into client bundles and are therefore treated as public — no secret may use those prefixes.
Provider misconfiguration	<code>DATABASE_PROVIDER=supabase</code> with a localhost URL is rejected at startup rather than silently serving a different database
Test-database guard	A <code>do_connect</code> listener on SQLAlchemy's <code>Engine</code> class with an exact-match allowlist of one target — a wrong database is <i>never dialled</i> , rather than merely discouraged. Collection additionally refuses the run with one clear message.

Guard	Behaviour
Release gate	<code>check-release-config.mjs</code> refuses to build the APK when a LAN backend URL is configured, so a laptop-dependent build cannot ship. Private-network, plain-http and loopback intelligence origins are rejected the same way.
JWKS unreachable	Returns 5xx , not 401 . A valid session must never be told it expired because we could not reach the identity provider — that sends the driver to a login screen that also cannot work.
Local-model host check	<code>looks_local()</code> refuses a non-loopback host unless deliberately overridden, so a driver's trip cannot be sent to a hosted endpoint by accident

11 Testing and quality

1,636 automated tests across three suites, plus two typecheck gates and a build gate

Suite	Framework	Result	Environment
Backend	pytest 8.3.4 + pytest-asyncio + httpx	976 passed 5 skipped	Isolated PostgreSQL 17 + PostGIS cluster on 127.0.0.1:55432 , target-guarded
Manager web	Vitest 3.2.7 + Testing Library + jsdom	150 passed	jsdom
Driver app	Vitest 4.1.11 + react-native-web	510 passed	jsdom over react-native-web
Type checking	tsc	clean	tsc -b --noEmit / tsc --noEmit
Lint	oxlint 1.79	clean	Manager web
Production build	Vite 8	succeeds	Manager web bundle

The 5 backend skips are named and legitimate: one non-Windows event-loop test, and four destructive migration tests gated behind RUN_DESTRUCTIVE_MIGRATION_TESTS=1 because they downgrade to base and drop every table.

The suites that pin down the properties this product depends on

Suite	What it pins down
test_db_target_guard.py	The run fails closed against any remote or unverified database, and refusals do not leak credentials
test_route_recommendation.py	Deterministic comparisons, evidence integrity, and that no confidence-like field reaches the wire
test_fuel_model.py	Unladen baseline, laden payload, elevation grade penalty
test_supabase_verifier.py	17 tests: algorithm pinning, iss / aud verification, unknown kid → 401, JWKS unreachable → 5xx, role never taken from the token
test_golden_path_e2e.py	One complete manager + driver operational cycle, end to end
test_concurrency.py	Behaviour under simultaneous writes
test_schema_drift.py	The ORM and the migration head agree
test_gemini_ai.py	Prompt-injection defence, medical-diagnosis refusal, rate limiter
MapScreen.test.tsx	Position truthfulness, honest clock aging, permission revocation
FleetMap.worker.build.test.ts	The production build actually emits the MapLibre worker
geminiAiHandler.test.ts	Reasoning-token stripping and per-mode prompt specialisation

A TEST CATEGORY MOST PROJECTS DO NOT HAVE

Several suites here assert what the system **must not** do: must not emit a confidence field, must not convert a provider outage into a score, must not let a stale reading lower a risk band, must not build an APK that depends on a laptop, must not dial a database other than the declared test target. Those are the tests that keep the honesty guarantees in section 1 true as the code changes.

12 Build, release and deployment

Artifacts, configuration surface and the reasoning behind each significant choice

Artifact	Toolchain and state
Android APK	EAS Build (hermetic). versionCode 9, build <code>3730b06b</code> , 71.29 MB, SHA-256 <code>228a1d3f...d437f9</code> . Passed a static security audit. Both preview and production profiles authenticate against the hosted platform.
Manager web bundle	Vite 8 production build, served as a static bundle
Intelligence service	Container image from <code>backend/Dockerfile</code> , described by <code>backend/render.yaml</code> ; health-checked on <code>/health</code> rather than <code>/ready</code> , so a brief database hiccup cannot make the platform kill a healthy process
Edge functions	Deployed to the Supabase project; <code>geminai-ai</code> verified HTTP 200
Local orchestration	<code>docker-compose.yml</code> and <code>scripts/db-start.ps1</code> / <code>db-stop.ps1</code> for the optional offline database

Configuration surface — selected keys

Variable	Default	Purpose
<code>APP_ENV</code>	development	Gates the placeholder-secret refusal
<code>DATABASE_PROVIDER</code>	supabase	<code>supabase</code> or <code>local</code> ; no automatic fallback between them
<code>DB_REQUIRE_SSL</code>	true	TLS to the database
<code>DB_CONNECT_TIMEOUT_SECONDS</code>	5	Fail fast rather than hang a request
<code>ROUTING_FALLBACK_URL</code>	router.project-osrm.org	Public OSRM; <code>ROUTING_PRIMARY_URL</code> takes a self-hosted instance
<code>ROUTING_TIMEOUT_SECONDS</code>	8.0	Routing budget
<code>WEATHER_PROVIDER_URL</code>	api.open-meteo.com	Weather provider; 6-second budget
<code>GEMINI_RPM_LIMIT</code>	10	Requests per minute ceiling
<code>AI_ALLOW_NON_LOCAL_HOST</code>	false	Structural guard on the local-model path
<code>LOGIN_RATE_LIMIT_PER_IP</code>	20	Per 60-second window; 10 per identifier
<code>CORS_ORIGINS</code>	—	Allowlist for the manager and Expo web origins
<code>EXPO_PUBLIC_INTELLIGENCE_BASE_URL</code>	—	Public https intelligence origin; private, http and loopback values are rejected at build time

Why each significant choice was made

Choice	Reasoning
Two apps, one backend	The manager works at a desk with several screens; the driver works one-handed in a cab with poor signal, sometimes at night. One responsive web app would compromise both.
Clients never touch the database	Capacity rules, document enforcement and the safety path are only unbypassable if there is exactly one way to write.
PostgreSQL + PostGIS	Corridor geometry, breadcrumb tracks and <code>ST_DWithin</code> stationary detection are first-class database operations, not application loops.
ap-south-1	Managed Postgres with PostGIS, edge functions and RLS, hosted in India — which matters for a government-sponsored logistics dataset.

Choice	Reasoning
psycopg 3, one driver	Serves both the async application engine and Alembic's sync engine from a single connection URL, so there is no second driver to keep in sync.
Python 3.11, not 3.14	The analytics-phase dependencies do not yet publish 3.14 wheels. <code>pyproject.toml</code> pins <code>>=3.11, <3.13</code> deliberately.
Start via <code>run.py</code>	On Windows the asyncio selector policy must be set before uvicorn creates its event loop, or every database call fails with a psycopg <code>InterfaceError</code> .
MapLibre + OpenStreetMap	Licence-free, self-hostable, and no per-map-load billing that would make a fleet console expensive to leave open all day.
Expo / React Native	One Android build from one TypeScript codebase, and <code>react-native-web</code> makes the same screens testable in CI without a device farm.
Stateless intelligence service	It holds no state of its own, so it can be scaled, moved or restarted freely — and every other workflow keeps working regardless.
Deterministic engines	Published constants can be argued with in review and defended in an inquiry. Every route decision carries the evidence it was made from — which is also exactly the dataset any future statistical model would have to beat.
No language model in the safety path	After a real incident, "the truck had not moved more than 50 m in 60 minutes and the driver did not answer in 30" is defensible. "The model scored 0.83" is not.

Scaling characteristics

Cost scales with trucks

Not with kilometres of road. All data sources are free and licence-clear; managed Postgres plus a container backend is the whole bill. No GPU, no on-prem server, no vehicle retrofit.

Geography is configuration

Only the hazard feeds are regional. The same schema, the same engines and the same clients serve Himalayan, Western Ghat and island logistics unchanged.

Horizontally scalable

The API is stateless and the intelligence service holds no state, so both scale by replica count. PostGIS indexes the geometry; corridor and stationary queries stay in the database.

Document provenance. Compiled 9 September 2026 from the working tree: `backend/requirements.txt`, `backend/pyproject.toml`, `manager-web/package.json`, `driver-app/package.json`, `backend/app/**`, `backend/alembic/versions/**`, `supabase/migrations/**`, `supabase/functions/**` and the engineering reports under `docs/`. Every coefficient, threshold, table name, endpoint and version string was read from source rather than restated from an earlier document.